Streaming Pipeline Benchmark Specification & Report

At-Least-Once vs. Idempotent Effectively-Once Persistence over Parquet Lake

Author: Raj Chhapariya | **Broker used:** local-emulator | **Date:** October 2026

1. Scope and What Was Measured

Kafka-style brokers cannot coordinate a two-phase commit with a file sink. When a consumer writes a Parquet batch and dies before its offset is recorded, the batch is read again on restart and its rows appear twice in the lake. This project measures that gap on 2,964,624 NYC Yellow Taxi trip records (January 2024) for two sinks: at-least-once (Mode A) and an offset-keyed idempotent sink (Mode B, called effectively-once because Kafka transactions do not cover a file sink).

Read this first: which broker produced these numbers

Every result in this report comes from the **local-emulator**, a disk-backed Kafka-style log in `pipeline/broker.py`, not from Redpanda. Docker was not available where the runs were made, so the Redpanda code path is written but has not been run end to end. Environment: Linux-6.8.0-138-generic-x86_64-with-glibc2.35; 13th Gen Intel(R) Core(TM) i7-13620H; 3,915 MB RAM; Python 3.10.12.

SOURCE ROWS	ACCEPTED ROWS	DUPLICATES, MODE A VS B	COMPACTION
2,964,624	2,869,979	3,531.3 vs 0.0	14.3x
NYC TLC, January 2024	96.81% pass validation	mean rows per run, boundary-crash variant	faster for the three queries

2. Pipeline and Delivery Modes

Stage	Implementation	Responsibility
Producer	`pipeline/producer.py`	Replays records at a target rate, validates each row, adds `row_id` and `produce_ts`, keys by `PULocationID` across 6 partitions.
Mode A	At-least-once	Writes the batch to `trips-p{P}-{uuid}.parquet`, then commits the offset to the broker. A kill in between replays the batch under a new name.
Mode B	Effectively-once	Names the file by offset range, writes to a temp file, renames it with `os.replace`, then records the range and the new offset in DuckDB in one transaction. On restart it resumes from the DuckDB offsets, deletes temp files and deletes finished files the manifest does not list.
Query layer	`pipeline/queries.py`	Three DuckDB aggregations over the date-partitioned Parquet files.

3. Correctness Under Forced Crashes

Each run stages 30,000 accepted rows (8,000 in the timer variant), then hard-kills the sink with SIGKILL 10 times at random moments, restarting it after every kill, and ends with an uninjected sink that drains the topic. Three seeds per row of the table. A *boundary crash* is the process dying between the file write and the offset record, with no cleanup; the plain kills land in that window only occasionally, so the injected variants make the window easier to hit.

Configuration	Crash variant	Runs	SIGKILLs delivered	Injected crashes	Mean duplicate rows	Max duplicate rows	Mean missing rows
at-least-once	sigkill-only	3	29	0	1,333.3	3,000	0.0
at-least-once	sigkill+boundary-crash	3	30	9	3,531.3	4,594	0.0
at-least-once	sigkill+boundary-crash+timer-batches	3	28	13	1,380.3	2,042	0.0
effectively-once	sigkill-only	3	29	0	0.0	0	0.0
effectively-once	sigkill+boundary-crash	3	30	10	0.0	0	0.0

effectively-once	sigkill+boundary-crash+timer-batches	3	28	12	0.0	0	0.0
effectively-once-no-reconcile	sigkill-only	3	30	0	0.0	0	0.0
effectively-once-no-reconcile	sigkill+boundary-crash	3	30	9	0.0	0	0.0
effectively-once-no-reconcile	sigkill+boundary-crash+timer-batches	3	28	10	862.3	1,491	0.0

Mode A duplicates rows in every injected variant and in some plain-kill runs, and never loses rows. Mode B produced no duplicates and no missing rows in any run. The row `effectively-once-no-reconcile` is Mode B built exactly as the build specification words it, which deletes temp files on restart but keeps a renamed file that the manifest does not list. With size-closed batches a replay reproduces the same file name and nothing is duplicated. With timer-closed batches the replay can cut a different offset range and both files stay: that variant leaked a mean of 862.3 duplicate rows per run, against 0.0 once unlisted files are deleted on restart.

4. Latency Versus Batch Size

The producer runs in a separate process at a fixed rate that the sink absorbs with near-zero lag, so queueing backlog is not part of the latency. A partition flushes after B rows or after the batch timer T, whichever comes first. Batches committed in the first 30 s of production and the partial batches at the end of a run are excluded. Sizes marked extra are not in the build specification; they show where the timer takes over.

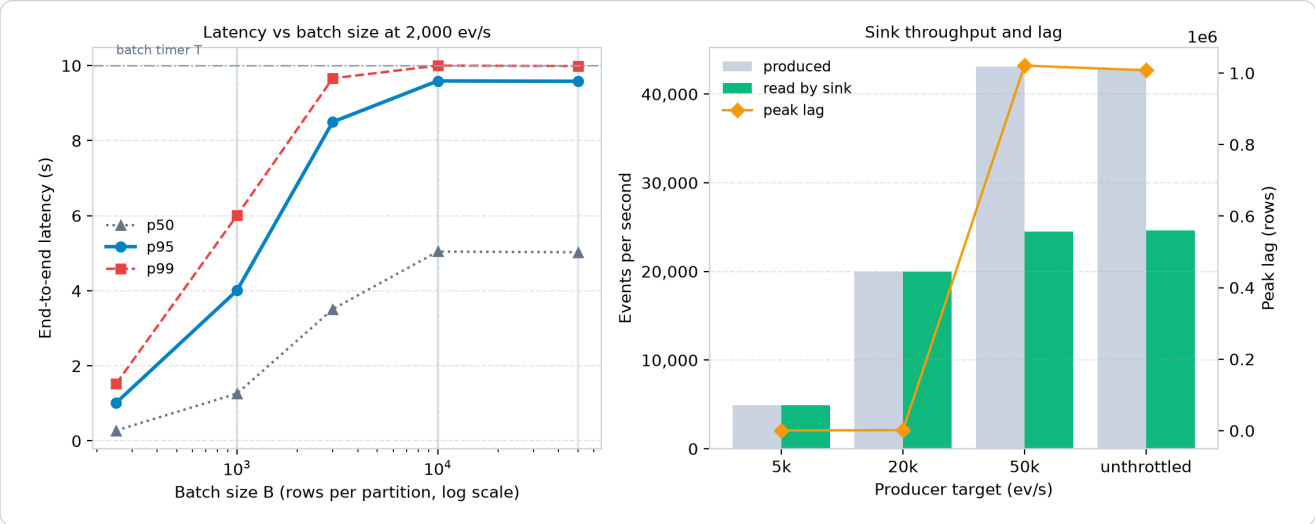


Figure 1: Left, p50, p95 and p99 end-to-end latency against batch size, with the batch timer drawn as a horizontal line. Right, producer and sink rates with peak lag for each target rate.

B (rows per partition)	p50 ms	p95 ms	p95 min to max	p99 ms	Batches closed by size / timer
250 (extra)	277	1,014	1,013 to 1,014	1,519	556 / 0
1,000	1,260	4,007	4,003 to 4,010	6,006	137 / 0
3,000 (extra)	3,510	8,499	8,497 to 8,503	9,660	33 / 19
10,000	5,045	9,591	9,563 to 9,601	9,999	0 / 40
50,000	5,025	9,581	9,579 to 9,593	9,987	0 / 40

Once B is large enough that a partition cannot fill before the timer fires, every batch is closed by the timer and latency stops depending on B. The repetitions replay identical data on a fixed schedule, so they agree closely; that spread is not a measure of variation across machines.

5. Throughput and Consumer Lag

Producer and sink run in separate processes. Lag is rows on the topic minus rows the sink has read, sampled every 0.5 s; the first 30 s of each run are excluded. The read path is the emulator's per-message index lookup, so these rates describe the emulator-backed sink and not Redpanda.

Target ev/s	Produced ev/s	Sink read ev/s (median)	Sink min to max	Peak lag (rows)	Lag when producer stopped
5,000	4,908	4,908	4,907 to 4,992	500	0

20,000	19,961	19,973	19,961 to 19,981	1,451	0
50,000	43,114	24,519	24,249 to 24,603	1,020,715	1,020,713
unthrottled	42,922	24,654	24,238 to 25,093	1,007,711	1,005,212

6. The Small-Files Problem and Compaction

The sink writes one file per micro-batch per partition, so the lake fills with small files. An offline job merges them. Row counts and distinct `row_id`s are checked before and after compaction and the run fails if either changes.

Metric	Before	After	Change
Rows in lake (measured)	40,000	40,000	unchanged
Parquet files	803	1	803x fewer
Size on disk (MB)	7.82	1.35	5.8x smaller
Query 1: hourly trips per zone (ms)	42.0	4.7	8.9x faster
Query 2: average fare per distance bucket (ms)	45.0	2.9	15.5x faster
Query 3: top 10 zone pairs (ms)	48.9	1.9	26.4x faster
All three queries (ms)	135.8	9.5	14.3x faster
Compaction run time (s)		0.69	

7. Input Validation

The producer checks each row against four rules before publishing. Of 2,964,624 source rows, 94,645 were rejected.

Rule	Rows rejected	Share of rows
zero_distance	60,371	2.036%
negative_fare	37,568	1.267%
dropoff_before_pickup	870	0.029%
outside_month	18	0.001%
total rejected	94,645	3.19%

8. Limitations

1. **No Redpanda run.** All numbers come from the local emulator. The Kafka-API path uses `assign` with stored offsets for Mode B and synchronous keyword commits for Mode A, and is covered only by tests against a stand-in consumer.
2. **Durability is not modelled.** A killed process does not lose the operating system page cache, so the crash tests exercise ordering, not fsync behaviour.
3. **The commit window is modelled.** Mode A waits 20 ms before the offset commit to stand in for the broker round trip. Plain-kill duplicate counts depend on that window.
4. **Partition skew.** Keying by `PULocationID` concentrates traffic on airport and Midtown zones. Salting the key, for example with `row_id mod 4`, spreads it at the cost of per-key ordering.
5. **Compaction is offline.** It is not safe against a concurrent sink, and a crash between writing the merged file and deleting its inputs leaves duplicate rows visible until it is rerun.

Resume line (numbers from the tables above): "Built an offset-keyed idempotent Parquet sink for a Kafka-style stream of 2,964,624 NYC taxi trips and tested it against a disk-backed broker emulator with hard kills and injected crashes: it produced 0.0 duplicate rows against a mean of 3,531.3 under at-least-once delivery, and a short-timer variant showed that a sink without orphan-file cleanup leaks 862.3 rows per run. Redpanda path implemented, not yet run."